

**Practical File
Of
Data Structures Laboratory
(UGCA1918)**



Submitted By: -

Bikkar Singh

Roll No. 2447072

Class: BCA 3rd Sem

Submitted to: -

Mr. Kuldeep Singh

Assistant professor

(Dept. of Computer Application)

**PUNJAB INSTITUTE OF MANAGEMENT AND TECHNOLOGY
(AFFILIATED TO IKG PUNJAB TECHNICAL UNIVERSITY, JALANDHAR)**

Computer Network Practical File Index Format

Sr. No.	Name of the Experiment/Program	Date of Submission	Page No.	Teacher's Signature	Remarks
1	Program for using Dynamic Functions (malloc(), calloc(), realloc() and free()) functions.		4		
2	Program to Insert, Delete and Traverse Elements in an Array		6		
3	Program to merge one dimensional arrays		9		
4	Program for addition and subtraction of two matrices.		11		
5	Program for implementing multiplication of two matrices		13		
6	Implement Linear Search using One and Two Dimensional Array		15		
7	Program for implementing selection sort.		18		
8	Program for implementing insertion sort		20		
9	Program for implementing quick sort.		22		
10	Program for implementing merge sort.		24		
11	Program to calculate length of the string using user defined function.		26		
12	Program to concatenate and compare two strings using user defined function.		27		
13	Program for using the concept of pointer to string.		29		
14	Program to reverse a sentence by recursion.		30		
15	Program to delete all repeated words in string.		31		
16	Program to find the number of vowels, consonants, digits and white space in a string.		33		
17	Program to find the length of the longest repeating sequence in a string.		35		

Computer Network Practical File Index Format

Sr. No.	Name of the Experiment/Program	Date of Submission	Page No.	Teacher's Signature	Remarks
18	Program to find highest and lowest frequency character in a string.		36		
19	Program for implementing Stack using array.		38		
20	Program for implementing Stack using pointer.		41		
21	Program for implementing multiple stack.		44		
22	Program for converting infix to postfix form.		49		
23	Program for implementing Queue using array.		52		
24	Program for dynamic implementation of queue.		55		
25	Program for implementing circular queue.		59		
26	Program for implementing dequeue.		62		
27	Program for implementing priority queue.		67		
28	Program for implementing Singly Linked list.		71		
29	Program for implementing Doubly Linked list		75		
30	Program for implementing Binary Search Tree.		81		
31	Program for Breadth First Search (BFS) for graph traversal.		85		
32	Program for Depth First Search (DFS) for graph traversal.		88		

1. Program for using Dynamic Memory Allocation Functions (malloc(), calloc(), realloc() and free())

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i, n;
    int *ptr;

    // Using malloc()
    printf("Enter number of elements: ");
    scanf("%d", &n);

    ptr = (int *)malloc(n * sizeof(int));

    if (ptr == NULL)
    {
        printf("Memory allocation failed");
        return 0;
    }

    printf("Enter elements (malloc):\n");
    for (i = 0; i < n; i++)
        scanf("%d", &ptr[i]);

    printf("Elements using malloc:\n");
    for (i = 0; i < n; i++)
        printf("%d ", ptr[i]);

    // Using realloc()
    ptr = (int *)realloc(ptr, (n + 2) * sizeof(int));

    printf("\nEnter 2 more elements:\n");
    for (i = n; i < n + 2; i++)
```

```
scanf("%d", &ptr[i]);

printf("Elements after realloc:\n");
for (i = 0; i < n + 2; i++)
    printf("%d ", ptr[i]);

// Free memory
free(ptr);

return 0;
}
```

Output

```
Enter number of elements: 3
Enter elements (malloc):
10 20 30
Elements using malloc:
10 20 30
Enter 2 more elements:
40 50
Elements after realloc:
10 20 30 40 50
```

2. Program to insert, delete and traverse an element from an array.

```
#include <stdio.h>

int main()
{
    int arr[100], n, i, pos, element, choice;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("\n1. Insert\n2. Delete\n3. Traverse\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            printf("Enter position to insert: ");
            scanf("%d", &pos);
            printf("Enter element to insert: ");
            scanf("%d", &element);

            for(i = n; i >= pos; i--)
                arr[i] = arr[i - 1];

            arr[pos - 1] = element;
            n++;

            printf("Array after insertion:\n");
            for(i = 0; i < n; i++)
                printf("%d ", arr[i]);
```

```
break;
```

```
case 2:
```

```
printf("Enter position to delete: ");
```

```
scanf("%d", &pos);
```

```
for(i = pos - 1; i < n - 1; i++)
```

```
    arr[i] = arr[i + 1];
```

```
n--;
```

```
printf("Array after deletion:\n");
```

```
for(i = 0; i < n; i++)
```

```
    printf("%d ", arr[i]);
```

```
break;
```

```
case 3:
```

```
printf("Array elements are:\n");
```

```
for(i = 0; i < n; i++)
```

```
    printf("%d ", arr[i]);
```

```
break;
```

```
default:
```

```
printf("Invalid choice");
```

```
}
```

```
return 0;
```

```
}
```

Output (Sample)

Insertion

Enter number of elements: 4

Enter array elements:

10 20 30 40

Enter your choice: 1

Enter position to insert: 3

Enter element to insert: 25

Array after insertion:

10 20 25 30 40

3. Program to merge one dimensional arrays

```
#include <stdio.h>

int main()
{
    int a[50], b[50], c[100];
    int n1, n2, i, j;

    printf("Enter number of elements in first array: ");
    scanf("%d", &n1);

    printf("Enter elements of first array:\n");
    for(i = 0; i < n1; i++)
        scanf("%d", &a[i]);

    printf("Enter number of elements in second array: ");
    scanf("%d", &n2);

    printf("Enter elements of second array:\n");
    for(i = 0; i < n2; i++)
        scanf("%d", &b[i]);

    // Copy first array elements
    for(i = 0; i < n1; i++)
        c[i] = a[i];

    // Copy second array elements
    for(j = 0; j < n2; j++)
        c[i + j] = b[j];

    printf("Merged array elements:\n");
    for(i = 0; i < n1 + n2; i++)
        printf("%d ", c[i]);

    return 0;
```

}

Output (Sample)

Enter number of elements in first array: 3

Enter elements of first array:

10 20 30

Enter number of elements in second array: 3

Enter elements of second array:

40 50 60

Merged array elements:

10 20 30 40 50 60

4. Program for addition and subtraction of two matrices.

```
#include <stdio.h>

int main()
{
    int a[10][10], b[10][10], sum[10][10], sub[10][10];
    int i, j, r, c;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &r, &c);

    printf("Enter elements of first matrix:\n");
    for(i = 0; i < r; i++)
        for(j = 0; j < c; j++)
            scanf("%d", &a[i][j]);

    printf("Enter elements of second matrix:\n");
    for(i = 0; i < r; i++)
        for(j = 0; j < c; j++)
            scanf("%d", &b[i][j]);

    // Addition of matrices
    for(i = 0; i < r; i++)
        for(j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];

    // Subtraction of matrices
    for(i = 0; i < r; i++)
        for(j = 0; j < c; j++)
            sub[i][j] = a[i][j] - b[i][j];

    printf("\nAddition of matrices:\n");
    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
```

```

        printf("%d ", sum[i][j]);
    printf("\n");
}

printf("\nSubtraction of matrices:\n");
for(i = 0; i < r; i++)
{
    for(j = 0; j < c; j++)
        printf("%d ", sub[i][j]);
    printf("\n");
}

return 0;
}

```

Output (Sample)

Enter number of rows and columns: 2 2

Enter elements of first matrix:

1 2

3 4

Enter elements of second matrix:

5 6

7 8

Addition of matrices:

6 8

10 12

Subtraction of matrices:

-4 -4

-4 -4

5. Program for implementing multiplication of two matrices

```
#include <stdio.h>

int main()
{
    int a[10][10], b[10][10], mul[10][10];
    int i, j, k;
    int r1, c1, r2, c2;

    printf("Enter rows and columns of first matrix: ");
    scanf("%d %d", &r1, &c1);

    printf("Enter rows and columns of second matrix: ");
    scanf("%d %d", &r2, &c2);

    if(c1 != r2)
    {
        printf("Matrix multiplication not possible");
        return 0;
    }

    printf("Enter elements of first matrix:\n");
    for(i = 0; i < r1; i++)
        for(j = 0; j < c1; j++)
            scanf("%d", &a[i][j]);

    printf("Enter elements of second matrix:\n");
    for(i = 0; i < r2; i++)
        for(j = 0; j < c2; j++)
            scanf("%d", &b[i][j]);

    // Initialize result matrix
    for(i = 0; i < r1; i++)
        for(j = 0; j < c2; j++)
            mul[i][j] = 0;
```

```

// Matrix multiplication
for(i = 0; i < r1; i++)
{
    for(j = 0; j < c2; j++)
    {
        for(k = 0; k < c1; k++)
        {
            mul[i][j] += a[i][k] * b[k][j];
        }
    }
}

printf("\nMultiplication of matrices:\n");
for(i = 0; i < r1; i++)
{
    for(j = 0; j < c2; j++)
        printf("%d ", mul[i][j]);
    printf("\n");
}

return 0;
}

```

Output (Sample)

Enter rows and columns of first matrix: 2 3

Enter rows and columns of second matrix: 3 2

Enter elements of first matrix:

1 2 3

4 5 6

Enter elements of second matrix:

7 8

9 10

11 12

Multiplication of matrices:

58 64

139 154

6. Implement linear search using one and two dimensional array.

```
#include <stdio.h>

int main()
{
    int a[50], n, i, key, found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++)
    {
        if(a[i] == key)
        {
            printf("Element found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }

    if(found == 0)
        printf("Element not found\n");

    return 0;
}
```

B) Linear Search in Two Dimensional Array

Program (2D Array)

```
#include <stdio.h>
```

```

int main()
{
    int a[10][10], r, c, i, j, key, found = 0;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &r, &c);

    printf("Enter matrix elements:\n");
    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
        {
            if(a[i][j] == key)
            {
                printf("Element found at row %d and column %d\n", i + 1, j +
1);
                found = 1;
                break;
            }
        }
    }

    if(found == 0)
        printf("Element not found\n");
}

```

```
    return 0;  
}
```

Output (1D Array)

Enter number of elements: 5

Enter array elements:

10 25 30 45 60

Enter element to search: 30

Element found at position 3

Output (2D Array)

Enter number of rows and columns: 2 3

Enter matrix elements:

1 2 3

4 5 6

Enter element to search: 5

Element found at row 2 and column 2

7. Program for implementing selection sort.

```
#include <stdio.h>

int main()
{
    int a[50], n, i, j, min, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    /* Selection Sort Logic */
    for(i = 0; i < n - 1; i++)
    {
        min = i;
        for(j = i + 1; j < n; j++)
        {
            if(a[j] < a[min])
            {
                min = j;
            }
        }

        /* Swapping */
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    printf("Sorted array using Selection Sort:\n");
```

```
    for(i = 0; i < n; i++)  
    {  
        printf("%d ", a[i]);  
    }  
  
    return 0;  
}
```

Output

Enter number of elements: 5

Enter array elements:

64 25 12 22 11

Sorted array using Selection Sort:

11 12 22 25 64

8. Program for implementing insertion sort.

```
#include <stdio.h>

int main()
{
    int a[50], n, i, j, key;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    /* Insertion Sort Logic */
    for(i = 1; i < n; i++)
    {
        key = a[i];
        j = i - 1;

        while(j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }

    printf("Sorted array in ascending order:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
}
```

```
    return 0;  
}
```

Enter 5 elements:

64

25

12

22

11

9. Program for implementing quick sort.

```
#include <stdio.h>

// Function to swap two elements
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Quick Sort function
void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(arr, low, high);
```

```

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Main function
int main()
{
    int arr[50], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

Output:

Enter number of elements: 6

Enter 6 elements:

45 23 89 12 56 34

Sorted array:

12 23 34 45 56 89

10. Program for implementing merge sort.

```
#include <stdio.h>

// Function to merge two sub-arrays
void merge(int arr[], int low, int mid, int high)
{
    int i = low, j = mid + 1, k = low;
    int temp[50];

    while(i <= mid && j <= high)
    {
        if(arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while(i <= mid)
        temp[k++] = arr[i++];

    while(j <= high)
        temp[k++] = arr[j++];

    for(i = low; i <= high; i++)
        arr[i] = temp[i];
}

// Merge Sort function
void mergeSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
```

```

        merge(arr, low, mid, high);
    }
}

// Main function
int main()
{
    int arr[50], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    mergeSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

Output:

Enter number of elements: 6

Enter 6 elements:

38 27 43 10 82 9

Sorted array:

9 10 27 38 43 82

11. Program to calculate length of the string using user defined function.

```
#include <stdio.h>

/* User defined function to find string length */
int stringLength(char str[])
{
    int i = 0;
    while (str[i] != '\0')
    {
        i++;
    }
    return i;
}

int main()
{
    char str[100];
    int length;

    printf("Enter a string: ");
    gets(str);

    length = stringLength(str);

    printf("Length of the string = %d", length);

    return 0;
}
```

Output:

Enter a string: Computer

Length of the string = 8

12. Program to concatenate and compare two strings using user defined function.

```
#include <stdio.h>

/* User defined function to concatenate strings */
void stringConcat(char str1[], char str2[])
{
    int i = 0, j = 0;

    while (str1[i] != '\0')
    {
        i++;
    }

    while (str2[j] != '\0')
    {
        str1[i] = str2[j];
        i++;
        j++;
    }
    str1[i] = '\0';
}

/* User defined function to compare strings */
int stringCompare(char str1[], char str2[])
{
    int i = 0;

    while (str1[i] == str2[i])
    {
        if (str1[i] == '\0')
            return 0;
        i++;
    }
    return 1;
}
```

```

}

int main()
{
    char str1[100], str2[100];
    int result;

    printf("Enter first string: ");
    gets(str1);

    printf("Enter second string: ");
    gets(str2);

    stringConcat(str1, str2);
    printf("Concatenated string = %s\n", str1);

    result = stringCompare(str1, str2);

    if (result == 0)
        printf("Both strings are equal");
    else
        printf("Both strings are not equal");

    return 0;
}

```

Output:

```

Enter first string: Hello
Enter second string: World
Concatenated string = HelloWorld
Both strings are not equal

```

13. Program for using the concept of pointer to string.

```
#include <stdio.h>

int main()
{
    char str[100];
    char *ptr;

    printf("Enter a string: ");
    gets(str);

    ptr = str;    /* Pointer pointing to string */

    printf("String using pointer: %s", ptr);
    return 0;
}
```

Output:

Enter a string: Pointer Example

String using pointer: Pointer Example

14. Program to reverse a sentence by recursion.

```
#include <stdio.h>

/* Recursive function to reverse a sentence */
void reverse()
{
    char ch;
    scanf("%c", &ch);

    if (ch != '\n')
    {
        reverse();
        printf("%c", ch);
    }
}

int main()
{
    printf("Enter a sentence: ");
    reverse();
    return 0;
}
```

Output:

```
Enter a sentence: Programming in C
C ni gnimmargorP
```

15. Program to delete all repeated words in string.

Program (C Language):

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[200], result[200];
    char *word;
    int found;

    printf("Enter a string: ");
    gets(str);

    result[0] = '\0';

    word = strtok(str, " ");
    while (word != NULL)
    {
        found = 0;
        if (strstr(result, word) != NULL)
        {
            found = 1;
        }

        if (!found)
        {
            strcat(result, word);
            strcat(result, " ");
        }

        word = strtok(NULL, " ");
    }

    printf("String after deleting repeated words:\n%s", result);
```

```
    return 0;  
}
```

Output:

Enter a string: this is is a test test string

String after deleting repeated words:

this is a test string

16. Program to find the number of vowels, consonants, digits and white space in a string.

```
#include <stdio.h>
#include <ctype.h>

int main()
{
    char str[200];
    int vowels = 0, consonants = 0, digits = 0, spaces = 0;
    int i;

    printf("Enter a string: ");
    gets(str);

    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] == ' ')
        {
            spaces++;
        }
        else if (isdigit(str[i]))
        {
            digits++;
        }
        else if (isalpha(str[i]))
        {
            if (str[i] == 'a' || str[i] == 'e' || str[i] == 'i' ||
                str[i] == 'o' || str[i] == 'u' ||
                str[i] == 'A' || str[i] == 'E' || str[i] == 'I' ||
                str[i] == 'O' || str[i] == 'U')
            {
                vowels++;
            }
            else
            {
                consonants++;
            }
        }
    }

    printf("Vowels: %d, Consonants: %d, Digits: %d, Spaces: %d\n", vowels, consonants, digits, spaces);
}
```

```
        consonants++;  
    }  
}  
  
printf("Vowels = %d\n", vowels);  
printf("Consonants = %d\n", consonants);  
printf("Digits = %d\n", digits);  
printf("White spaces = %d", spaces);  
  
return 0;  
}
```

Output:

Enter a string: Hello World 123

Vowels = 3

Consonants = 7

Digits = 3

White spaces = 2

17. Program to find the length of the longest repeating sequence in a string.

```
#include <stdio.h>

int main()
{
    char str[200];
    int i, count = 1, max = 1;

    printf("Enter a string: ");
    gets(str);

    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] == str[i + 1])
        {
            count++;
            if (count > max)
            {
                max = count;
            }
        }
        else
        {
            count = 1;
        }
    }

    printf("Length of the longest repeating sequence = %d", max);

    return 0;
}
```

Output:

Enter a string: aaabbccccdde

Length of the longest repeating sequence = 4

18. Program to find highest and lowest frequency character in a string.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[200];
    int freq[256] = {0};
    int i;
    char maxChar, minChar;
    int max, min;

    printf("Enter a string: ");
    gets(str);

    for (i = 0; str[i] != '\0'; i++)
    {
        freq[(int)str[i]]++;
    }

    max = 0;
    min = strlen(str);

    for (i = 0; i < 256; i++)
    {
        if (freq[i] != 0)
        {
            if (freq[i] > max)
            {
                max = freq[i];
                maxChar = i;
            }
            if (freq[i] < min)
            {
                min = freq[i];
            }
        }
    }
}
```

```
        minChar = i;
    }
}

printf("Highest frequency character = %c (%d times)\n", maxChar, max);
printf("Lowest frequency character = %c (%d times)", minChar, min);

return 0;
}
```

Output:

Enter a string: sample string

Highest frequency character = s (2 times)

Lowest frequency character = a (1 times)

19. Program for implementing Stack using array.

```
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

/* Push operation */
void push()
{
    int item;
    if (top == MAX - 1)
    {
        printf("Stack Overflow\n");
    }
    else
    {
        printf("Enter element to push: ");
        scanf("%d", &item);
        top++;
        stack[top] = item;
        printf("Element pushed successfully\n");
    }
}

/* Pop operation */
void pop()
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
    }
    else
    {
        printf("Popped element = %d\n", stack[top]);
    }
}
```

```

        top--;
    }
}

/* Display stack */
void display()
{
    int i;
    if (top == -1)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Stack elements:\n");
        for (i = top; i >= 0; i--)
        {
            printf("%d\n", stack[i]);
        }
    }
}

int main()
{
    int choice;
    while (1)
    {
        printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: push(); break;
            case 2: pop(); break;

```

```

        case 3: display(); break;
        case 4: return 0;
        default: printf("Invalid choice\n");
    }
}
}

```

Output

```

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element to push: 10
Element pushed successfully
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element to push: 20
Element pushed successfully
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements:
20
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped element = 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements:
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4

```

20. Program for implementing Stack using pointer.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int main()
{
    int *stack;
    int top = -1;
    int choice, item, i;

    stack = (int *)malloc(MAX * sizeof(int));

    if (stack == NULL)
    {
        printf("Memory allocation failed\n");
        return 0;
    }

    while (1)
    {
        printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                if (top == MAX - 1)
                {
                    printf("Stack Overflow\n");
                }
                else
                {
```

```

        printf("Enter element to push: ");
        scanf("%d", &item);
        top++;
        *(stack + top) = item;
        printf("Element pushed successfully\n");
    }
    break;

case 2:
    if (top == -1)
    {
        printf("Stack Underflow\n");
    }
    else
    {
        printf("Popped element = %d\n", *(stack + top));
        top--;
    }
    break;

case 3:
    if (top == -1)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Stack elements:\n");
        for (i = top; i >= 0; i--)
        {
            printf("%d\n", *(stack + i));
        }
    }
    break;

```

```
        case 4:
            free(stack);
            return 0;

        default:
            printf("Invalid choice\n");
    }
}
```

Output:

1. Push

2. Pop

3. Display

4. Exit

Enter your choice: 1

Enter element to push: 5

Element pushed successfully

Enter your choice: 1

Enter element to push: 15

Element pushed successfully

Enter your choice: 3

Stack elements:

15

5

Enter your choice: 2

Popped element = 15

21. Program for implementing multiple stack.

```
#include <stdio.h>

#define MAX 10

int stack[MAX];

int top1 = -1, top2 = MAX;

void push1(int item)
{
    if (top1 + 1 == top2)
        printf("Stack1 Overflow\n");
    else
    {
        top1++;
        stack[top1] = item;
        printf("Element pushed to Stack1\n");
    }
}

void push2(int item)
{
    if (top2 - 1 == top1)
        printf("Stack2 Overflow\n");
    else
    {
        top2--;
        stack[top2] = item;
        printf("Element pushed to Stack2\n");
    }
}

void pop1()
{
    if (top1 == -1)
        printf("Stack1 Underflow\n");
```

```

else
{
    printf("Popped from Stack1 = %d\n", stack[top1]);
    top1--;
}
}

void pop2()
{
    if (top2 == MAX)
        printf("Stack2 Underflow\n");
    else
    {
        printf("Popped from Stack2 = %d\n", stack[top2]);
        top2++;
    }
}

void display1()
{
    int i;
    if (top1 == -1)
        printf("Stack1 is empty\n");
    else
    {
        printf("Stack1 elements:\n");
        for (i = top1; i >= 0; i--)
            printf("%d\n", stack[i]);
    }
}

void display2()
{
    int i;
    if (top2 == MAX)

```

```

        printf("Stack2 is empty\n");
    else
    {
        printf("Stack2 elements:\n");
        for (i = top2; i < MAX; i++)
            printf("%d\n", stack[i]);
    }
}

int main()
{
    int choice, stackNo, item;

    while (1)
    {
        printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter stack number (1 or 2): ");
                scanf("%d", &stackNo);
                printf("Enter element: ");
                scanf("%d", &item);
                if (stackNo == 1) push1(item);
                else if (stackNo == 2) push2(item);
                else printf("Invalid stack number\n");
                break;

            case 2:
                printf("Enter stack number (1 or 2): ");
                scanf("%d", &stackNo);
                if (stackNo == 1) pop1();

```

```
        else if (stackNo == 2) pop2();
        else printf("Invalid stack number\n");
        break;

    case 3:
        printf("Enter stack number (1 or 2): ");
        scanf("%d", &stackNo);
        if (stackNo == 1) display1();
        else if (stackNo == 2) display2();
        else printf("Invalid stack number\n");
        break;

    case 4:
        return 0;

    default:
        printf("Invalid choice\n");
    }
}
}
```

Output:

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter stack number (1 or 2): 1

Enter element: 10

Element pushed to Stack1

Enter your choice: 1

Enter stack number (1 or 2): 2

Enter element: 20

Element pushed to Stack2

Enter your choice: 3

Enter stack number (1 or 2): 1

Stack1 elements:

10

Enter your choice: 3

Enter stack number (1 or 2): 2

Stack2 elements:

20

22. Program for converting infix to postfix form.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 50

char stack[MAX];
int top = -1;

void push(char c)
{
    stack[++top] = c;
}

char pop()
{
    if (top == -1)
        return '\0';
    return stack[top--];
}

int precedence(char c)
{
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

int main()
{
    char infix[MAX], postfix[MAX];
    char ch;
    int i = 0, k = 0;
```

```

printf("Enter infix expression: ");
gets(infix);

while ((ch = infix[i++]) != '\0')
{
    if (isalnum(ch))
    {
        postfix[k++] = ch;
    }
    else if (ch == '(')
    {
        push(ch);
    }
    else if (ch == ')')
    {
        while (top != -1 && stack[top] != '(')
            postfix[k++] = pop();
        pop(); // remove '('
    }
    else // operator
    {
        while (top != -1 && precedence(stack[top]) >= precedence(ch))
            postfix[k++] = pop();
        push(ch);
    }
}

while (top != -1)
    postfix[k++] = pop();

postfix[k] = '\0';

printf("Postfix expression: %s", postfix);

```

```
return 0;  
}
```

Output:

Enter infix expression: A+B*C

Postfix expression: ABC*+

23. Program for implementing Queue using array.

```
#include <stdio.h>

#define MAX 5

int queue[MAX];

int front = -1, rear = -1;

/* Enqueue operation */
void enqueue()
{
    int item;
    if (rear == MAX - 1)
    {
        printf("Queue Overflow\n");
    }
    else
    {
        printf("Enter element to enqueue: ");
        scanf("%d", &item);
        rear++;
        queue[rear] = item;
        printf("Element enqueued successfully\n");
    }
}

/* Dequeue operation */
void dequeue()
{
    if (front == rear)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        front++;
    }
}
```

```

        printf("Dequeued element = %d\n", queue[front]);
    }
}

/* Display queue */
void display()
{
    int i;
    if (front == rear)
    {
        printf("Queue is empty\n");
    }
    else
    {
        printf("Queue elements:\n");
        for (i = front + 1; i <= rear; i++)
        {
            printf("%d\n", queue[i]);
        }
    }
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: enqueue(); break;

```

```
        case 2: dequeue(); break;
        case 3: display(); break;
        case 4: return 0;
        default: printf("Invalid choice\n");
    }
}
}
```

Output:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter element to enqueue: 10

Element enqueued successfully

Enter your choice: 1

Enter element to enqueue: 20

Element enqueued successfully

Enter your choice: 3

Queue elements:

10

20

Enter your choice: 2

Dequeued element = 10

24. Program for dynamic implementation of queue.

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

/* Enqueue operation */
void enqueue()
{
    int item;
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL)
    {
        printf("Memory allocation failed\n");
        return;
    }

    printf("Enter element to enqueue: ");
    scanf("%d", &item);
    newNode->data = item;
    newNode->next = NULL;

    if (rear == NULL) // empty queue
    {
        front = rear = newNode;
    }
    else
```

```

    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("Element enqueued successfully\n");
}

/* Dequeue operation */
void dequeue()
{
    if (front == NULL)
    {
        printf("Queue Underflow\n");
        return;
    }

    struct Node *temp = front;
    front = front->next;

    if (front == NULL)
        rear = NULL;

    printf("Dequeued element = %d\n", temp->data);
    free(temp);
}

/* Display queue */
void display()
{
    struct Node *temp = front;
    if (temp == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

```

```

    }

    printf("Queue elements:\n");
    while (temp != NULL)
    {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: return 0;
            default: printf("Invalid choice\n");
        }
    }
}

```

Output:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter element to enqueue: 10

Element enqueued successfully

Enter your choice: 1

Enter element to enqueue: 20

Element enqueued successfully

Enter your choice: 3

Queue elements:

10

20

Enter your choice: 2

Dequeued element = 10

25. Program for implementing circular queue.

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

/* Enqueue operation */
void enqueue()
{
    int item;
    if ((rear + 1) % MAX == front)
    {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter element to enqueue: ");
    scanf("%d", &item);

    if (front == -1) // first element
        front = 0;

    rear = (rear + 1) % MAX;
    queue[rear] = item;

    printf("Element enqueued successfully\n");
}

/* Dequeue operation */
void dequeue()
{
    if (front == -1)
    {
        printf("Queue Underflow\n");
    }
}
```

```

        return;
    }

    printf("Dequeued element = %d\n", queue[front]);

    if (front == rear) // only one element
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

/* Display queue */
void display()
{
    int i;
    if (front == -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements:\n");

    i = front;
    while (1)
    {
        printf("%d\n", queue[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
}

int main()
{

```

```

int choice;

while (1)
{
    printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1: enqueue(); break;
        case 2: dequeue(); break;
        case 3: display(); break;
        case 4: return 0;
        default: printf("Invalid choice\n");
    }
}
}

```

Output:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter element to enqueue: 10

Element enqueued successfully

Enter your choice: 1

Enter element to enqueue: 20

Element enqueued successfully

Enter your choice: 3

Queue elements:

10

20

Enter your choice: 2

Dequeued element = 10

26. Program for implementing dequeue.

```
#include <stdio.h>

#define MAX 5

int dequeue[MAX];
int front = -1, rear = -1;

/* Insert at front */
void insertFront(int item)
{
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
    {
        printf("Dequeue Overflow\n");
        return;
    }

    if (front == -1) // empty
        front = rear = 0;
    else if (front == 0)
        front = MAX - 1;
    else
        front--;

    dequeue[front] = item;
    printf("Element inserted at front: %d\n", item);
}

/* Insert at rear */
void insertRear(int item)
{
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
    {
        printf("Dequeue Overflow\n");
        return;
    }
}
```

```

    if (rear == -1) // empty
        front = rear = 0;
    else if (rear == MAX - 1)
        rear = 0;
    else
        rear++;

    dequeue[rear] = item;
    printf("Element inserted at rear: %d\n", item);
}

/* Delete from front */
void deleteFront()
{
    if (front == -1)
    {
        printf("Dequeue Underflow\n");
        return;
    }

    printf("Deleted element from front: %d\n", dequeue[front]);

    if (front == rear) // only one element
        front = rear = -1;
    else if (front == MAX - 1)
        front = 0;
    else
        front++;
}

/* Delete from rear */
void deleteRear()
{
    if (rear == -1)

```

```

{
    printf("Dequeue Underflow\n");
    return;
}

printf("Deleted element from rear: %d\n", dequeue[rear]);

if (front == rear) // only one element
    front = rear = -1;
else if (rear == 0)
    rear = MAX - 1;
else
    rear--;
}

/* Display dequeue */
void display()
{
    int i;
    if (front == -1)
    {
        printf("Dequeue is empty\n");
        return;
    }

    printf("Dequeue elements:\n");
    i = front;
    while (1)
    {
        printf("%d\n", dequeue[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
}

```

```

int main()
{
    int choice, item;

    while (1)
    {
        printf("\n1. Insert Front\n2. Insert Rear\n3. Delete Front\n4. Delete
Rear\n5. Display\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter element to insert at front: ");
                scanf("%d", &item);
                insertFront(item);
                break;
            case 2:
                printf("Enter element to insert at rear: ");
                scanf("%d", &item);
                insertRear(item);
                break;
            case 3: deleteFront(); break;
            case 4: deleteRear(); break;
            case 5: display(); break;
            case 6: return 0;
            default: printf("Invalid choice\n");
        }
    }
}

```

Output:

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter your choice: 1

Enter element to insert at front: 10

Element inserted at front: 10

Enter your choice: 2

Enter element to insert at rear: 20

Element inserted at rear: 20

Enter your choice: 5

Dequeue elements:

10

20

Enter your choice: 3

Deleted element from front: 10

27. Program for implementing priority queue.

```
#include <stdio.h>

#define MAX 5

int queue[MAX], priority[MAX];
int front = -1, rear = -1;

/* Enqueue operation */
void enqueue()
{
    int item, prio;
    if (rear == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter element to enqueue: ");
    scanf("%d", &item);
    printf("Enter its priority (higher number = higher priority): ");
    scanf("%d", &prio);

    rear++;
    queue[rear] = item;
    priority[rear] = prio;

    if (front == -1)
        front = 0;

    printf("Element enqueued successfully\n");
}

/* Dequeue operation */
void dequeue()
{

```

```

if (front == -1)
{
    printf("Queue Underflow\n");
    return;
}

int i, max = front, item, prio;

for (i = front + 1; i <= rear; i++)
{
    if (priority[i] > priority[max])
        max = i;
}

item = queue[max];
prio = priority[max];

printf("Dequeued element = %d with priority = %d\n", item, prio);

for (i = max; i < rear; i++)
{
    queue[i] = queue[i + 1];
    priority[i] = priority[i + 1];
}

rear--;

if (rear < front)
    front = -1;
}

/* Display queue */
void display()
{
    int i;

```

```

    if (front == -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements with priorities:\n");
    for (i = front; i <= rear; i++)
    {
        printf("Element: %d, Priority: %d\n", queue[i], priority[i]);
    }
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: return 0;
            default: printf("Invalid choice\n");
        }
    }
}

```

Output:

- 1. Enqueue**
- 2. Dequeue**
- 3. Display**
- 4. Exit**

Enter your choice: 1

Enter element to enqueue: 10

Enter its priority (higher number = higher priority): 2

Element enqueued successfully

Enter your choice: 1

Enter element to enqueue: 20

Enter its priority (higher number = higher priority): 5

Element enqueued successfully

Enter your choice: 3

Queue elements with priorities:

Element: 10, Priority: 2

Element: 20, Priority: 5

Enter your choice: 2

Dequeued element = 20 with priority = 5

28. Program for implementing Singly Linked list.

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct Node
{
    int data;
    struct Node *next;
};

struct Node *head = NULL;

/* Insert at beginning */
void insertAtBeginning()
{
    int item;
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode)
    {
        printf("Memory allocation failed\n");
        return;
    }
    printf("Enter element to insert: ");
    scanf("%d", &item);
    newNode->data = item;
    newNode->next = head;
    head = newNode;
    printf("Element inserted at beginning\n");
}

/* Insert at end */
void insertAtEnd()
{
    int item;
```

```

    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode)
    {
        printf("Memory allocation failed\n");
        return;
    }
    printf("Enter element to insert: ");
    scanf("%d", &item);
    newNode->data = item;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        struct Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Element inserted at end\n");
}

/* Delete from beginning */
void deleteFromBeginning()
{
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;

```

```

    head = head->next;
    printf("Deleted element = %d\n", temp->data);
    free(temp);
}

/* Display linked list */
void display()
{
    struct Node *temp = head;
    if (temp == NULL)
    {
        printf("List is empty\n");
        return;
    }

    printf("Linked list elements:\n");
    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Insert at Beginning\n2. Insert at End\n3. Delete from Beginning\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
    }
}

```

```
switch (choice)
{
    case 1: insertAtBeginning(); break;
    case 2: insertAtEnd(); break;
    case 3: deleteFromBeginning(); break;
    case 4: display(); break;
    case 5: return 0;
    default: printf("Invalid choice\n");
}
}
```

Output:

1. Insert at Beginning
2. Insert at End
3. Delete from Beginning
4. Display
5. Exit

Enter your choice: 1

Enter element to insert: 10

Element inserted at beginning

Enter your choice: 2

Enter element to insert: 20

Element inserted at end

Enter your choice: 4

Linked list elements:

10 -> 20 -> NULL

Enter your choice: 3

Deleted element = 10

29. Program for implementing Doubly Linked list

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};

struct Node *head = NULL;

/* Insert at beginning */
void insertAtBeginning()
{
    int item;
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode)
    {
        printf("Memory allocation failed\n");
        return;
    }
    printf("Enter element to insert: ");
    scanf("%d", &item);
    newNode->data = item;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
        head->prev = newNode;

    head = newNode;
    printf("Element inserted at beginning\n");
}
```

```

}

/* Insert at end */
void insertAtEnd()
{
    int item;
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode)
    {
        printf("Memory allocation failed\n");
        return;
    }
    printf("Enter element to insert: ");
    scanf("%d", &item);
    newNode->data = item;
    newNode->next = NULL;

    if (head == NULL)
    {
        newNode->prev = NULL;
        head = newNode;
    }
    else
    {
        struct Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }
    printf("Element inserted at end\n");
}

/* Delete from beginning */
void deleteFromBeginning()

```

```

{
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;
    head = head->next;

    if (head != NULL)
        head->prev = NULL;

    printf("Deleted element = %d\n", temp->data);
    free(temp);
}

/* Delete from end */
void deleteFromEnd()
{
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    if (temp->prev != NULL)
        temp->prev->next = NULL;
    else
        head = NULL;
}

```

```

    printf("Deleted element = %d\n", temp->data);
    free(temp);
}

/* Display forward */
void displayForward()
{
    struct Node *temp = head;
    if (temp == NULL)
    {
        printf("List is empty\n");
        return;
    }

    printf("Doubly linked list (forward):\n");
    while (temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

/* Display backward */
void displayBackward()
{
    struct Node *temp = head;
    if (temp == NULL)
    {
        printf("List is empty\n");
        return;
    }

    while (temp->next != NULL)
        temp = temp->next;

```

```

    printf("Doubly linked list (backward):\n");
    while (temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Insert at Beginning\n2. Insert at End\n3. Delete from
Beginning\n4. Delete from End\n5. Display Forward\n6. Display Backward\n7.
Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: insertAtBeginning(); break;
            case 2: insertAtEnd(); break;
            case 3: deleteFromBeginning(); break;
            case 4: deleteFromEnd(); break;
            case 5: displayForward(); break;
            case 6: displayBackward(); break;
            case 7: return 0;
            default: printf("Invalid choice\n");
        }
    }
}

```

Output:

1. Insert at Beginning
2. Insert at End
3. Delete from Beginning
4. Delete from End
5. Display Forward
6. Display Backward
7. Exit

Enter your choice: 1

Enter element to insert: 10

Element inserted at beginning

Enter your choice: 2

Enter element to insert: 20

Element inserted at end

Enter your choice: 5

Doubly linked list (forward):

10 <-> 20 <-> NULL

Enter your choice: 6

Doubly linked list (backward):

20 <-> 10 <-> NULL

30. Program for implementing Binary Search Tree.

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};

/* Create new node */
struct Node* createNode(int item)
{
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = item;
    newNode->left = newNode->right = NULL;
    return newNode;
}

/* Insert into BST */
struct Node* insert(struct Node* root, int item)
{
    if (root == NULL)
        return createNode(item);

    if (item < root->data)
        root->left = insert(root->left, item);
    else if (item > root->data)
        root->right = insert(root->right, item);

    return root;
}
```

```

/* Inorder traversal */
void inorder(struct Node* root)
{
    if (root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

/* Search in BST */
int search(struct Node* root, int key)
{
    if (root == NULL)
        return 0;

    if (key == root->data)
        return 1;
    else if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}

int main()
{
    struct Node* root = NULL;
    int choice, item, key;

    while (1)
    {
        printf("\n1. Insert\n2. Inorder Traversal\n3. Search\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
    }
}

```

```

switch (choice)
{
    case 1:
        printf("Enter element to insert: ");
        scanf("%d", &item);
        root = insert(root, item);
        printf("Element inserted successfully\n");
        break;

    case 2:
        printf("Inorder Traversal: ");
        inorder(root);
        printf("\n");
        break;

    case 3:
        printf("Enter element to search: ");
        scanf("%d", &key);
        if (search(root, key))
            printf("Element found in BST\n");
        else
            printf("Element not found in BST\n");
        break;

    case 4:
        return 0;

    default:
        printf("Invalid choice\n");
}
}
}

```

Output:

1. Insert

2. Inorder Traversal

3. Search

4. Exit

Enter your choice: 1

Enter element to insert: 50

Element inserted successfully

Enter your choice: 1

Enter element to insert: 30

Element inserted successfully

Enter your choice: 1

Enter element to insert: 70

Element inserted successfully

Enter your choice: 2

Inorder Traversal: 30 50 70

Enter your choice: 3

Enter element to search: 70

Element found in BST

31. Program for Breadth First Search (BFS) for graph traversal.

```
#include <stdio.h>

#define MAX 10

int queue[MAX], front = -1, rear = -1;
int visited[MAX];

/* Insert into queue */
void enqueue(int value)
{
    if (rear == MAX - 1)
        return;
    if (front == -1)
        front = 0;
    queue[++rear] = value;
}

/* Delete from queue */
int dequeue()
{
    if (front == -1 || front > rear)
        return -1;
    return queue[front++];
}

int main()
{
    int n, start;
    int graph[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
```

```

for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        scanf("%d", &graph[i][j]);

for (int i = 0; i < n; i++)
    visited[i] = 0;

printf("Enter starting vertex (0 to %d): ", n - 1);
scanf("%d", &start);

enqueue(start);
visited[start] = 1;

printf("BFS Traversal: ");
while (front <= rear)
{
    int current = dequeue();
    printf("%d ", current);

    for (int i = 0; i < n; i++)
    {
        if (graph[current][i] == 1 && visited[i] == 0)
        {
            enqueue(i);
            visited[i] = 1;
        }
    }
}

return 0;
}

```

Output:

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

Enter starting vertex (0 to 3): 0

BFS Traversal: 0 1 2 3

32. Program for Depth First Search (DFS) for graph traversal.

```
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX], visited[MAX];
int n;

/* DFS function */
void dfs(int vertex)
{
    int i;
    printf("%d ", vertex);
    visited[vertex] = 1;

    for (i = 0; i < n; i++)
    {
        if (graph[vertex][i] == 1 && visited[i] == 0)
        {
            dfs(i);
        }
    }
}

int main()
{
    int start, i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);
```

```
for (i = 0; i < n; i++)
    visited[i] = 0;

printf("Enter starting vertex (0 to %d): ", n - 1);
scanf("%d", &start);

printf("DFS Traversal: ");
dfs(start);

return 0;
}
```

Output:

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

Enter starting vertex (0 to 3): 0

DFS Traversal: 0 1 3 2